

Overload sets as function parameters

Contents

1. [Motivation](#)
2. [History](#)
3. [Proposal](#)
 - 3.1. [Implementation details](#)
 - 3.2. [Constraining on overload set](#)
 - 3.3. [Deduction rules](#)
 - 3.3.1. [Deducing from an object](#)
 - 3.3.2. [Deducing from a name of a function](#)
 - 3.3.2.1. [Deducing from a name that cannot be found by unqualified lookup](#)
 - 3.3.3. [Deducing from partial class member access](#)
 - 3.3.4. [Deducing from a type-member access](#)
 - 3.3.5. [Deducing from an operator-function-id](#)
 - 3.3.6. [Deducing from a type name](#)
 - 3.3.7. [Summary of deduction rules](#)
 - 3.4. [Standard library functions, now and future](#)
 - 3.5. [Alternative spellings](#)
 - 3.6. [Other potential avenues](#)
4. [Acknowledgements](#)
5. [References](#)

§ 1. Motivation

Calling a function in C++ is pretty fundamental. But abstracting that function call, the most basic generalization, doesn't always work out neatly.

Anytime today that we can write:

```
namespace N {
    struct X { ... };
    X getX();
}

foo(N::getX()); // for any 'foo'
```

We might want to also be able to write:

```
template <typename F>
void algorithm(F f, X x) {
    f(x);
}

algorithm(foo, N::getX()); // for that same 'foo'
```

These are, conceptually, very similar. But there are many cases where the former code compiles and runs without issue but the latter fails:

1. `foo` could be a function that takes default arguments
2. `foo` could be a function template

3. `foo` could name an overload set
4. `foo` could be a function that was only found by ADL
5. `foo` in unqualified lookup could have found one function, and so the call to `algorithm()` succeeds, but the ADL `foo` wasn't found and so the call within `algorithm()` fails. Worst case, the `foo` found by unqualified lookup is actually a viable candidate, so the wrong function gets called.
6. `foo` could be the name of a non-static member function, or non-static member function template, and we are invoking it from within a non-static member function.
7. `foo` could be a partial member access, something like `obj.func`, which is valid to spell only in the context of invoking the function.
8. `foo` could name a function in the standard library, which we are not allowed to take a pointer to (this restriction made more explicit in light of [P0551](#) and [P0921](#)).

The only solution to this problem today, outside of trafficking exclusively in function objects, is to manually wrap `foo` in a lambda - and probably be quite vigilant about doing so:

```
algorithm(getX(), [&](auto&& ...args)
    noexcept(noexcept(foo(std::forward<decltype(args)>(args)...)))
-> decltype(foo(std::forward<decltype(args)>(args)...)) {
    return foo(std::forward<decltype(args)>(args)...);
});
```

which is usually seen in the wild in macro form:

```
#define FWD(x) static_cast<decltype(x)&&>(x)
#define RETURNS(expr) noexcept(noexcept(expr)) -> decltype(expr) { return expr; }
#define OVERLOADS_OF(name) [&](auto&& ...args) RETURNS(name(FWD(args)...))

algorithm(getX(), OVERLOADS_OF(foo));
```

This can be found, for instance, in [Boost.HOF](#) as `BOOST_HOF_LIFT`, and in a recent blog of Andrzej Krzemiński's on [this topic](#).

However, this is a pretty unsatisfactory solution: we rely on a macro. Or even if not, we have to be vigilant about manually wrapping each and every function at every call site. Why "every"? Because otherwise, we might write code that works today but ends up being very brittle, easily broken. Consider a fairly trivial example:

```
// some library
void do_something(int );

// some user
std::invoke(do_something, 42);
```

This works fine today. But the library that provided `do_something` might someday wish to make some improvements. Maybe add a defaulted second argument to `do_something`. Or a new overload. Or turn `do_something` into a function template. Any number of changes that would not change the meaning of code directly invoking `do_something` with an `int`. The kinds of changes detailed in [P0921](#). All of these changes would break the above user code - even though it compiles today. So *even here*, it would be better had the user written:

```
std::invoke(OVERLOADS_OF(do_something), 42);
```

or, at the very least:

```
std::invoke([](int i){ do_something(i); }, 42);
```

And that's simply too much to ask of the user - it's too much to have to think about it! We're asking the user to, every time, put a bunch of seemingly unnecessary annotation on every call site. After all, in the context of being invoked by a single argument, these two *should be* equivalent:

<pre>[&](auto&& arg) noexcept(noexcept(foo(FWD(arg))))</pre>	<pre>foo</pre>
--	----------------

```

-> decltype(foo(FWD(arg)))
{
    return foo(FWD(arg));
}

```

But they are unfortunately not equivalent today, so we have to write the stuff on the left. But the idea we want to express is *just* to invoke a function by name. Having to explicitly list all of the arguments twice - once in the parameter list, once in the call expression of the lambda body (or even four times in the *noexcept-specifier* and *trailing-return-type*) - does not aid the reader in any way.

This is a real problem today. There are many, many higher-order functions that exist in C++ code. They exist in the standard library algorithms, they exist in user-provided libraries. With the adoption of [Ranges](#), we will get many more - both in terms of algorithms, projections on algorithms, and views. Higher-order functions are ubiquitous, and incredibly useful. Unfortunately, user problems in trying to pass callables to them are just as ubiquitous [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#) [\[6\]](#) [\[7\]](#) ... It's important to give users a solution to this problem, that preferably is not reaching for a macro like `OVERLOADS_OF` or having to manually write a lambda (possibly inefficiently).

Being able to write just the function name, as opposed to having to list the arguments and the body, is sometimes referred to as "[point-free programming](#)", a term more commonly used in languages like Haskell. Being able to write point-free in the context of C++ means being able to pass in names and have that *just work* regardless of what the name happens to refer to. It means that any time `foo(x)` works that `algorithm(foo, x)` could be made to work also.

Titus Winters in a [recent C++Now talk](#) described the overload set as the atom of C++ API design. And yet, we cannot even pass this atom into other atoms with reaching for lambdas, forwarding references, and trailing return types.

We can do better.

§ 2. History

This problem has a long history attached to it, with two different tacks explored.

[P0119](#) proposed a syntax-free, "just make it work" approach: synthesize a lambda if, based on a heuristic, it's likely the intended behavior (e.g. if the name nominates an overload set). Unfortunately, this solution fails, as demonstrated in [P0382](#).

[N3617](#) (also [EWG65](#)) and later [P0834](#) proposed syntax for the caller to use to explicitly synthesize an overload set:

```

algorithm(getX(), []foo);
std::invoke([]do_something, 42);

```

This was rejected by EWG in the [Albuquerque](#), but it is a syntax that has many benefits. It is point-free and terse enough as to be practically invisible. It would still place the onus on the user to avoid brittleness at each call site, but it is a very manageable burden. That discussion did conclude with this poll:

Are we interested in a core language feature for packing concrete overload sets?

SF	F	N	A	SA
3	4	14	0	1

[P0573](#) wouldn't have directly solved this problem, but would have at least made writing the direct lambda less burdensome. It was also rejected in [Albuquerque](#), and did not even try to solve the point-free problem.

Despite this long history, we believe that this is a problem that needs to be solved. It is unreasonably difficult today to pass a function into another function. The increased emphasis on disallowing users from taking pointers to standard library functions and function templates directly pushes the issue. A significant portion of the discussion of [P0798](#) in LEWG in [Rapperswil](#) was about the problem of passing overload sets into functions - because the paper would simply introduce more places where users may want to do such a thing. Notably, LEWG took these polls:

Assuming the language gets fixed so we can pass overload sets to callables, we want something like this (either as a general monad syntax or specifically in optional).

SF	F	N	A	SA
12	4	0	0	0

We want something like this, even if the language is not fixed with respect to overload sets.

SF	F	N	A	SA
5	4	3	1	1

This is a problem that needs solving, and it has to be solved at the language level.

§ 3. Proposal

The community tried to solve this problem without adding any syntax, and it tried to solve this problem by adding syntax on the caller's site. We propose a new approach: adding syntax on the function declaration itself. That is, rather than having overload sets as function *arguments* we instead have overload sets as function *parameters*.

We propose a new magic library type, `std::overload_set<T>` (see [alternative spellings](#)) that will deduce an overload set, and we will call this new object an *overload set object*. The fundamental goal driving the design of `overload_set` is that anywhere `foo(a, b, c)` works today, `std::overload_set(foo)(a, b, c)` should also work and have the same meaning:

```
void foo(int);           // #1
void foo(int, int);      // #2

template <typename F, typename... Args>
void algorithm(std::overload_set<F> f, Args... args) {
    f(args...);
}

foo(1);                  // calls #1
foo(2, 3);               // calls #2
algorithm(foo, 1);        // ok: calls #1
algorithm(foo, 2, 3);     // ok: calls #2

std::overload_set f = foo; // note: deduction guide magic
f(1);                    // ok: calls #1
f(2, 3);                 // ok: calls #2
```

The difference is that, instead of burdening the caller to annotate each and every function, it's up to the API to do that annotation. There are many more callers than callees, so this seems like it places the burden on the correct party. APIs that use `overload_set` would be very friendly for users, and would allow for a point-free style.

For legacy APIs that do not use `overload_set` for their callables, `overload_set` can be used on the call site as a substitute for the macro. This is admittedly much longer than the `[]` proposed in P0834/N3617, but it is at least substantially better than a macro. And legacy APIs can be easily transitioned to take advantage of this feature by simply adding [another overload](#).

§ 3.1. Implementation details

An `overload_set<T>` contains a `T` and has a call operator that forwards to `T`'s call operator. Its copy and move constructors and assignment operators are defaulted, its other constructors are unspecified.

A hypothetical implementation might look like:

```
template <typename T>
class overload_set {
    T f;
public:
    overload_set(/* unspecified */);

    overload_set(overload_set const&) = default;
    overload_set(overload_set&&) = default;
    overload_set& operator=(overload_set const&) = default;
    overload_set& operator=(overload_set&&) = default;
    ~overload_set() = default;
```

```

template <typename... Us>
invoke_result_t<F&, Us...> operator()(Us&&... us) &
    noexcept(is_nothrow_invocable_v<F&, Us...>);

template <typename... Us>
invoke_result_t<F const&, Us...> operator()(Us&&... us) const&
    noexcept(is_nothrow_invocable_v<F const&, Us...>);

template <typename... Us>
invoke_result_t<F, Us...> operator()(Us&&... us) &&
    noexcept(is_nothrow_invocable_v<F, Us...>);

template <typename... Us>
invoke_result_t<F const, Us...> operator()(Us&&... us) const&&
    noexcept(is_nothrow_invocable_v<F const, Us...>);
};

```

With appropriate deduction guides. An open question is whether or not an `overload_set<T>` should be (explicitly or implicitly) convertible to a `T`.

3.2. Constraining on `overload_set`

Since `overload_set<T>` simply contains a `T`, and its call operators are based on `T`s, this interacts very well with how template constraints are written:

```

// SFINAE constraint has the desired effect
template <typename F, enable_if_t<is_invocable_v<F&, int>, int> = 0>
void foo(overload_set<F> f) {
    f(42);
}

// trailing-return constrained has the desired effect
template <typename F>
auto foo(overload_set<F> f) -> decltype(f(17)) {
    return f(17);
}

// concept has the desired effect
template <LvalueInvocable<int> F>
void bar(overload_set<F> f) {
    f(63);
}

```

3.3. Deduction rules

The annotation `overload_set` will trigger new deduction rules based not just on the type of the argument it is being deduced from, but also on its *name*, and even the token sequence.

There are many cases we're proposing, each will synthesize a slightly different function object. We will go over these proposed deduction rules in detail, and then [summarize them](#).

3.3.1. Deducing from an object

If the argument is an object, whether it is a function object or a pointer (or reference) to function or pointer to member, then `overload_set<T>` deduces `T` as the object's type:

```

auto square = [](int i){ return i * i; };

template <typename T> void deduce(std::overload_set<T> f);
deduce(square); // deduce T as decltype(square)

```

The function parameter `f` will have the same underlying type as `square`. There is no synthesis of a new lambda in this scenario, we are just copying the lambda.

An `overload_set` can be deduced from an object if that object is callable. It must be either a pointer or reference to function or member function, or have class type with at least one declared `operator()` or one declared conversion function which can create a surrogate call function as per [\[over.call.object\]](#). For any other type, `overload_set` cannot be deduced:

```
deduce(42); // error
deduce(new int); // error
deduce(&square); // error
deduce("hi"s); // error
```

We also propose that deducing an `overload_set` whose template type parameter is an rvalue reference to cv-unqualified template parameter behaves as a forwarding reference. Since `overload_set<T>` is simply a `T`, `overload_set<T&>` is likewise simply a `T&`. This allows for overload set objects that are actually references:

```
// copy
template <typename T> auto f_copy(overload_set<T> f) { return f; }

// const copy
template <typename T> auto f_const(overload_set<T const> f) { return f; }

// lvalue reference
template <typename T> auto f_lref(overload_set<T&> f) { return f; }

// forwarding reference
template <typename T> auto f_fref(overload_set<T&&> f) { return f; }

struct Counter {
    int i;
    int operator() { return ++i; }
};

Counter c{0};
c();
assert(c.i == 1);

f_copy(c());
assert(c.i == 1);
f_const(c()); // ill-formed, Counter::operator() isn't const
f_lref(c());
assert(c.i == 2);
f_fref(c()); // calls f_fref<Counter&>, because forwarding reference
assert(c.i == 3);
```

This allows the API to avoid copying if it so desires.

This rule gives the appearance of the `&` simply being in the wrong place. But there is a difference between `overload_set<T&>` (an overload set object which is a reference to a callable object) and `overload_set<T&&>` (a reference to an overload set object).

3.3.2. Deducing from a name of a function

If the argument is the (qualified or unqualified) name of a function, function template, static member function or static member function template, or an overload set containing any number of either, then `overload_set<T>` deduces `T` as a synthesized lambda in the style of `OVERLOADS_OF`:

```
[](auto&& ...args) noexcept(noexcept(name(FWD(args)...)))
-> decltype(name(FWD(args)...)) {
    return name(FWD(args)...);
}
```

Qualified names synthesize a lambda that makes a qualified call. Unqualified names synthesize a lambda that makes an unqualified call. Note that this lambda has no capture - we're just calling functions by name.

This is true even if name lookup finds *one single* function:

```
int square(int i) {
    return i * i;
}

// note that square is not an object here, it's the name of a function.
// the underlying type of f is a lambda, not a function pointer
auto f = std::overload_set(square);
```

The reason for this is we want to have ADL still work:

```
void g(int);
void h(int);
void h(double);
namespace N {
    struct X { };

    void g(X);
    void h(X);
}

auto over_g = std::overload_set(g);
auto over_h = std::overload_set(h);
over_g(N::X{}); // ok, calls N::g... not an error trying to invoke ::g
over_h(N::X{}); // ok, calls N::h
```

Note that this is only the case if we deduce by *name*. If we had instead passed in a pointer to `g`, we would be in the object case described in the previous section:

```
auto ptr_g = std::overload_set(&g);
ptr_g(0); // ok
ptr_g(N::X{}); // error, no conversion from N::X to int

auto ptr_h = std::overload_set(&h); // error: unresolved overloaded function
```

The lookup set is frozen at the time of the construction of the overload set object, in the same way it would be had we done it manually:

```
void foo(int);
auto f1 = std::overload_set(foo);
void foo(double);
auto f2 = std::overload_set(foo);

f1(2.0); // calls foo(int)
f2(2.0); // calls foo(double)
```

This works with function templates or qualified names:

```
namespace N {
    template <typename T> T twice(T);
}

std::invoke(overload_set(N::twice), 1); // calls N::twice<int>
std::invoke(overload_set(N::twice), "hi"s); // calls N::twice<std::string>
```

It is unspecified whether or not multiple overload set objects created for the same name in the same context have the same type.

This works even with function templates where one template parameter is explicitly provided but the rest are deduced:

```
template <typename T, typename U>
T convert_to(U);

// this is okay
auto to_int = overload_set(convert_to<int>);
to_int(1); // ok: calls convert_to<int, int>
to_int(2.0); // ok: calls convert_to<int, double>

// but not this
auto convert = overload_set(convert_to);
convert(1); // error: can't deduce T
convert<int>(1); // error: the 'int' would apply to the argument of convert
// there's no 'passthrough' of template parameters
```

It is an open question as to whether or not parenthesized unqualified names should disable ADL (and get early diagnosed typos) or not:

```
void foo(int);
namespace N {
    struct X { };
    void foo(X);
}

auto f = std::overload_set((foo)); // NB: parenthesized
f(N::X{}); // ok or error?
```

Does that call succeed (ADL finding `N::foo`) or does the synthesized function behave as if `((foo)(FWD(args)...))`, which would not consider ADL, and hence fail with no matching overload?

3.3.2.1. Deducing from a name that cannot be found by unqualified lookup

Consider a slight variation from an example from the previous section:

```
void g(int);
void h(int);
void h(double);
namespace N {
    struct X { };

7   void f(X);
    void g(X);
    void h(X);
}

12  auto over_f = std::overload_set(f);
    auto over_g = std::overload_set(g);
    auto over_h = std::overload_set(h);

16  over_f(N::X{});
    over_g(N::X{});
    over_h(N::X{});
```

Should this work? In the previous section, we introduced a rule based on synthesizing a lambda if name lookup finds a function or function template or overload set... but on line 12 here, name lookup on `f` finds nothing. Can we really ignore that?

Perhaps `f` was a typo, and the programmer really meant `g` and we would be doing them a disservice if we do not diagnose at the point of its use on line 12.

On the other hand, a user trying to pass the name `f` into a function would write this code:


```
auto over_f = [](auto e) { return f(e); };
over_f(N:X{});
```

which would diagnose at the point of call rather than at the point of the declaration of the lambda if there was no such function `f`.

The motivation of this paper is very much to allow for the separation of the function from its arguments, and since in the original example `f(N:X{})` would work (by finding `N::f` via ADL), we firmly believe that `std::overload_set(f)(N:X{})` should work as well.

In order to achieve that goal, we amend the previous stated rule to also synthesize the same kind of lambda from an unqualified name for which lookup finds nothing.

We can safely reject any attempt to use a qualified name for which we cannot find any candidates.

3.3.3. Deducing from partial class member access

If the argument is a partial class member access, of the forms `x.y`, `x->y`, or, in the context of a member function, `y`, where `y` is the name of a member function, member function template, overload set of member functions, or an object that is callable in some way, then `overload_set<T>` deduces `T` as a synthesized type which captures the class object or pointer.

The rule here is that `overload_set(obj.mem)` will copy `obj`, just as `overload_set(ptr->mem)` will copy `ptr`. This gives the user control over how the object is captured to invoke the member function. If capture-by-copy is desired, use the dot access syntax. If capture-by-reference is desired, then either take a pointer to your object (i.e. `overload_set(&obj).mem`) or wrap it in a reference (i.e.

`overload_set(std::ref(obj).mem)`).

```
std::string s = "hello";
std::overload_set f = s.size;           // f holds a copy of s, i.e. by value
std::overload_set g1 = (&s)->size;      // g1 holds a copy of &s, i.e. by reference
std::overload_set g2 = ref(s).size;     // g2 also holds a reference

s += ", world";
assert(f() == 5);
assert(g1() == 12);
assert(g2() == 12);
```

This rule is necessary to cover the two situations where we can directly invoke a function by name, but cannot stash the name to be invoked later: invoking a non-static member function from within the body of a non-static member function, and normal invocation of member functions on objects:

```
struct X {
    void foo();

    void bar() {
        foo();           // ok
        std::overload_set(foo)(); // proposed ok
    }
};

X x;
x.foo();                // ok
std::overload_set(x.foo)(); // proposed ok
```

Such partial member syntax is ill-formed today, which seems wasteful. This syntax is available in Python and is widely used. The desire to write such code is the entire motivation for [P0356](#):

```
struct Strategy { double process(std::string, std::string, double, double); };

std::unique_ptr<Strategy> createStrategy();

// p0356: note that this suffers from all of the same problems
// through this paper regarding API brittleness, due to the named
```

```
// pointer to member function
auto f = std::bind_front(&Strategy::process, createStrategy());

// proposed: strictly better than the above: more expressive
// of user intent, in addition to simply working in more cases
auto g = std::overload_set(createStrategy()->process);
```

Given a general output function iterator, you could, for instance, specify `push_back()` directly instead of using `back_inserter()`:

```
// today
std::transform(src.begin(), src.end(),
               std::back_inserter(dst),
               f);

// proposed
std::transform(src.begin(), src.end(),
               function_output(std::ref(dst).push_back),
               f);
```

3.3.4. Deducing from a type-member access

When it comes to member functions, we depart somewhat from our initially stated goal of having the validity of `foo(a, b, c)` imply the validity of `std::overload_set(foo)(a, b, c)`. While member functions are conceptually functions that take, as their first argument, an instance of the class type, that is not the way they are spelled (despite `INVOKE`). Moreover, we cannot use `&Class::mem` as a launching point for a special deduction rule, since that already is a pointer to member function and would be covered by the object rules.

However, invoking member functions in such a way is still highly useful today, and will become even more highly in demand with the adoption of Ranges and its heavy use of projections. The motivating example for projections in [N4128](#) was:

```
std::sort(a, std::less<>(), &employee::last);
auto p = std::lower_bound(a, "Parent", std::less<>(), &employee::last);
```

Which is great. And `last` is a non-static data member, there are no problems whatsoever. But what if `last` is an overloaded function or a function template? Such cases are not infrequent, and should be just as easy to use!

We propose a novel syntax to cover these cases: `Type.member`. Such syntax is ill-formed today, and has an intuitive meaning: invoke the member `member` given an instance of type `Type`. In other words, `Type.member` is syntax for invoking a non-static member function/data set whereas `Type::member` is syntax for invoking a static member function/data set.

That is:

```
struct X {
    template <typename T>
    void bar(T) const;
    void bar(double, double) const;
};

std::overload_set bar = X.bar;

X x;
bar(x, 1);           // calls X::bar<int>
bar(&x, 'x');        // calls X::bar<char>
bar(std::ref(x), 2.0, 3.0); // calls X::bar(double, double)
```

The underlying type of `bar` behaves similarly to having synthesized:

```
struct overloads_x_bar {
    template <typename T, typename... Args>
        requires std::is_base_of_v<X, std::uncvref_t<T>>
    auto operator()(T&& x, Args&&... args) const
```

```

    RETURNS(FWD(x).bar(FWD(args)...))

template <typename T, typename... Args>
    requires std::is_base_of_v<X,
        std::uncvref_t<decltype(*std::declval<T>())>>
    auto operator()(T&& x, Args&&... args) const
        RETURNS((*FWD(x)).bar(FWD(args)...))

template <typename T, typename... Args>
    requires std::is_base_of_v<X, T>
    auto operator()(std::reference_wrapper<T> x, Args&&... args) const
        RETURNS(x.get().bar(FWD(args)...))
};

overloads_x_bar bar;

```

In this way, `std::overload_set(X.bar)` is a strictly superior alternative to `std::mem_fn(&X::bar)` and effectively obsoletes it:

Defaulted arguments	
<pre> struct C { int f(int, int=4); } obj; std::mem_fn(&C::f)(obj, 1); // error std::mem_fn(&C::f)(obj, 1, 2); // ok, obj.f(1, 2) </pre>	<pre> struct C { int f(int, int=4); } obj; std::overload_set(C.f)(obj, 1); // ok, obj.f(1, 4); std::overload_set(C.f)(obj, 1, 2); // ok, obj.f(1, 2); </pre>
Overloads	
<pre> struct C { int g(int); int g(int, int); } obj; std::mem_fn(&C::g); // error already // technically, okay, for some definition of okay std::mem_fn(static_cast<int(C::*)(int)>(&C::g))(obj, 1); </pre>	<pre> struct C { int g(int); int g(int, int); } obj; std::overload_set(C.g)(obj, 1); // ok, obj.g(1) std::overload_set(C.g)(obj, 1, 2); // ok, obj.g(1, 2) </pre>
Templates	
<pre> struct C { template <typename T> void h(T); } obj; std::mem_fn(&C::h); // error already std::mem_fn(&C::h<int>, obj, 1); // ok, obj.h<int>(1) std::mem_fn(&C::h<int>, obj, '1'); // ok, but // manual template deduction picked the wrong type </pre>	<pre> struct C { template <typename T> void h(T); } obj; std::overload_set(C.h)(obj, 1); // ok, obj.h<int>(1) std::overload_set(C.h)(obj, '1'); // ok, obj.h<char>('1') </pre>

In the case where the notation `Type.member` nominates a data member (whether static or non-static), we can simply fallback to using the object deduction from the pointer to member `&Type::member`. In these cases, there are no issues with overloading or template deduction - since there can just be one member.

Getting back to the initial examples from Ranges. Assuming projections would be implemented using this proposal, we would write:

```

std::sort(a, std::less<>(), employee.last);
auto p = std::lower_bound(a, "Parent", std::less<>(), employee.last);

```

And then just... not have to worry about anything. It just works.

§ 3.3.5. Deducing from an *operator-function-id*

Going one step further from the previous cases, we want to further allow the synthesis of a lambda overload set from the name of an *operator-function-id*. This will synthesize a unary, binary, or hybrid callable depending on the operator in question.

Many of these cases are already covered by preexisting function objects in `<functional>`. `std::overload_set(operator<)` is `std::less()`, `std::overload_set(operator==)` is `std::equal_to()`, and so forth. So why do we need them?

The main argument is a question of readability. Everyone is familiar with the operators, they jump out more than names of objects. Not everyone is familiar with all of the equivalent names - very few are familiar with all of them.

As initially suggested in P0119, we propose a shorthand parenthesized form for these operators as well.

```
std::sort(a, std::greater());
```

```
// assuming sort() can be specified to take an
// overload_set<Compare>
std::sort(values, operator>);
std::sort(values, (>));

// equivalent to
std::sort(values, [](auto&& a, auto&& b)
-> decltype(FWD(a) > FWD(b)) {
    retur FWD(a) > FWD(b);
});
```

Combining with the previous sections examples, this gives us:

```
// works only if the pointer to member &employee::last
// can be formed
std::sort(a, std::less<>(), &employee::last);
auto p = std::lower_bound(a, "Parent", std::less<>(),
    &employee::last);
```

```
// works regardless of how 'last' is implemented
std::sort(a, (<), employee.last);
auto p = std::lower_bound(a, "Parent", (<), employee.last);
```

The synthesized lambda for the call operator would behave equivalently to `std::invoke`. That is:

```
auto inv = std::overload_set(operator());
inv(a, b, c); // equivalent to std::invoke(a, b, c);
```

§ 3.3.6. Deducing from a type name

One final place situation we want to handle is when the function or overload set we're invoking is actually a constructor. In this case, the name isn't the name of a function or a function template, it's the name of a type:

```
struct X {
    X(int);
};

// with Ranges today
std::vector<X> makeXs(std::vector<int> const& input) {
    return input | view::transform([](int i){ return X(i); });
}
```

The lambda `[](int i){ return X(i); }` follows the same pattern as can be seen throughout the paper. In the same way that we've been able to replace this lambda with an overload set object for functions and operators and class member access, we'd also like to replace this for types. Writing `X(1)` is a valid expression, so writing `std::overload_set(X)(1)` should be as well:

```
// Proposed
std::vector<X> makeXs(std::vector<int> const& input) {
```

```

    return input | view::transform(X);
}

```

A type name very much behaves as if it were an overload set that, when invoked, produces an instance of that type. The `emplace()` functions are very much forwarding arguments to that type function. Being able to deduce an overload set object from a type name just cements that model.

3.3.7. Summary of deduction rules

The following table goes through all the rules presented above. For each row, on the left is some declaration

`std::overload_set name = tokens;` and on the right is an declaration for the type `T` that would be synthesized for `std::overload_set<T>` in this deduction.

Declaration	Equivalent synthesized type
Objects	
<pre> auto square = [](int i) { return i * i; }; std::overload_set a = square; </pre>	<pre> using T_a = decltype(square); </pre>
<pre> void (*f)() = +[] { /* ... */ }; std::overload_set b = f; </pre>	<pre> using T_b = void(*)(); </pre>
Function names	
<pre> void foo(int); void foo(int, int); std::overload_set c = foo; </pre>	<pre> struct T_c { template <typename... Args> auto operator()(Args&&... args) const RETURNS(foo(FWD(args)...)) }; </pre>
<pre> std::overload_set d = std::max; </pre>	<pre> struct T_d { template <typename... Args> auto operator()(Args&&... args) const RETURNS(std::max(FWD(args)...)) }; </pre>
<pre> struct X { static void bar(); static void bar(int); template <typename T> static void bar(T); }; std::overload_set e = X::bar; </pre>	<pre> struct T_e { template <typename... Args> auto operator()(Args&&... args) const RETURNS(X::bar(FWD(args)...)) }; </pre>
Partial class access	
<pre> std::string s = "copy"; std::overload_set f = s.size; </pre>	<pre> struct T_f { std::string _s; template <typename... Args> auto operator()(Args&&... args) & RETURNS(_s.size(FWD(args)...)) template <typename... Args> auto operator()(Args&&... args) && RETURNS(std::move(_s).size(FWD(args)...)) </pre>

	<pre>// + two more overloads };</pre>
<pre>std::string s = "by_ref"; std::overload_set g = (&s)->size;</pre>	<pre>struct T_g { std::string* _s; template <typename... Args> auto operator()(Args&&... args) const RETURNS(_s->size(FWD(args)...)) };</pre>
<pre>std::string s = "by_ref"; std::overload_set h = std::ref(s).append;</pre>	<pre>struct T_h { std::reference_wrapper<std::string> _s; template <typename... Args> auto operator()(Args&&... args) const RETURNS(_s.get().append(FWD(args)...)) };</pre>
Class type member access	
<pre>std::overload_set i = std::string.size;</pre>	<pre>struct T_i { template <typename T, typename... Args> requires std::is_base_of_v<std::string, std::uncvref_t<T>> auto operator()(T&& x, Args&&... args) const RETURNS(FWD(x).size(FWD(args)...)) template <typename T, typename... Args> requires std::is_base_of_v<std::string, std::uncvref_t<decltype(*std::declval<T>())>> auto operator()(T&& x, Args&&... args) const RETURNS((*FWD(x)).size(FWD(args)...)) template <typename T, typename... Args> requires std::is_base_of_v<std::string, T> auto operator()(std::reference_wrapper<T> x, Args&&... args) const RETURNS(x.get().size(FWD(args)...)) };</pre>
<pre>struct Person { int id; std::string name; }; std::overload_set j = Person.id;</pre>	<pre>struct T_j { int Person::* pmd; template <typename... Args> auto operator()(Args&&... args) const RETURNS(std::invoke(pmd, std::forward<Args>(args)...)) };</pre>
Operators	
<pre>std::overload_set k = operator<;</pre>	<pre>struct T_k { template <typename T, typename U> auto operator()(T&& t, U&& u) const RETURNS(FWD(t) < FWD(u)) };</pre>
<pre>std::overload_set m = operator!;</pre>	<pre>struct T_m { template <typename T> auto operator()(T&& t) const</pre>

	<pre> RETURNS(!FWD(t)) }; </pre>
<pre> std::overload_set n = operator-; </pre>	<pre> struct T_n { template <typename T> auto operator()(T&& t) const RETURNS(-FWD(t)) template <typename T, typename U> auto operator()(T&& t, U&& u) const RETURNS(FWD(t) - FWD(u)) }; </pre>
Type names	
<pre> std::overload_set o = std::string; </pre>	<pre> struct T_o { template <typename... Args> auto operator()(Args&&... args) const RETURNS(std::string(FWD(args)...)) }; </pre>

3.4. Standard library functions, now and future

One of the papers mentioned earlier, P0798, proposed a handful of new member functions for `std::optional`. Currently these just all deduce their arguments in the same way as all other standard library function templates. But if we, instead, specify them to deduce an overload set parameter:

```

template <typename T>
struct optional {
    template <Invocable<T const&> F>
    auto map(std::overload_set<F> f) const& -> optional<std::invoke_result_t<F&, T const&>>
    {
        if (!*this) {
            return nullopt;
        } else {
            return f(**this);
        }
    }
};

```

Then any reasonable usage of it would just work, do the right thing, and be a usage that would be blessed by P0921.

```

std::optional<std::string> f();

f().map(&std::string::size);    // okay, would compile, but not allowed
f().map(std::size<std::string>); // likewise
f().map(std::size);            // okay with this proposal, ill-formed today
f().map(size);                // okay with this proposal (would call std::size via ADL), ill-formed today
f().map(std::string.size);     // okay with this proposal, ill-formed today

```

Otherwise, the **only** approved way to get this behavior would be:

```

f().map([](auto&& s){ return s.size(); });

```

Which is a lot to write for a simple task. Instead, people will consistently reach for the shorter, more brittle solutions. Let's just solve the problem instead.

As far as preexisting standard library function templates go. We cannot simply *change* the ones that exist. But, we do allow ourselves to add overloads. We could easily do that, and take advantage of the proposed deduction rules of `overload_set` to not change any behavior:

```
// new overload
template <class InputIt, class UnaryPredicate>
constexpr InputIt find_if(InputIt first, InputIt last, std::overload_set<UnaryPredicate> p) {
    return find_if(first, last, std::ref(p));
}

// existing overload
template <class InputIt, class UnaryPredicate>
constexpr InputIt find_if(InputIt first, InputIt last, UnaryPredicate p) {
    // same as it ever was
}
```

All existing, viable calls to `find_if` will continue to call the existing overload. It is a better match. But a wide variety of new calls to `find_if()` suddenly become possible where they weren't before! This isn't perfect - we still allow users to take pointers to functions and member functions if those happen to work today. But it's a big step in better usability for everyone.

§ 3.5. Alternative spellings

The name `overload_set` makes a judgement about what is being passed, which may not actually be an overload set. We are very open to considering other names. Unfortunately `function` is taken, but some alternatives might be:

- `invocable<T>`
- `callable<T>`
- `[] T` (along the lines of N3617/P0834)

§ 3.6. Other potential avenues

Over the last several standards, C++ has been adding more and more support for a more functionally oriented programming style. Lambdas and then generic lambdas. Algebraic data types (`tuple` , `optional` , `variant`).

But there are a few things that are still sufficiently difficult to do in the language to the point where we don't even think about them as approaches. One such difficulty is function composition. How do you compose two functions? We can use F#'s syntax for [function composition](#): `<<` and `>>` (references and forwarding omitted for brevity):

```
// g, then f
template <typename F, typename G>
auto operator<<(overload_set<F> f, overload_set<G> g) {
    return [=](auto... xs) -> decltype(f(g(xs...)))
        return f(g(xs...));
}

// f, then g
template <typename F, typename G>
auto operator>>(overload_set<F> f, overload_set<G> g) {
    return g << f;
}
```

This let's us replace another of the standard library's function objects. We have `std::not_fn()` , which takes one function object and produces a new function objects whose effects are negating the original one. But with function composition, we don't need a special function like `not_fn` . We can just compose with negation:

C++17	With composition
<pre>namespace std { template<class F> unspecified not_fn(F&& f); } auto g = std::not_fn(f);</pre>	<pre>// We could use the preexisting function object auto g1 = f >> std::negate(); // Or we could use the new ability to use operator-function-ids auto g2 = f >> operator!; // Or we could use the new ability to use parenthesized operators</pre>


```
auto g3 = f >> (!);

// Or we could spell out the !
auto g4 = f >> (not);
```

The composition choices aren't necessarily shorter than using `not_fn`. But it's one less thing to have to be aware of, one less thing to keep track of. You just need to know about negation. It's arguably much more expressive.

Composition isn't the only thing we could do with multiple functions - we could also combine multiple operations together. For instance, `f(x) * g(x)` could be expressed as `(f * g)(x)`:

```
template <typename F, typename G>
auto operator*(overload_set<F> f, overload_set<G> g) {
    return [=](auto x) -> decltype(f(x) * g(x))
        return f(x) * g(x);
}
```

Perhaps also `overload_set<T>` could curry by default. Or provide a way to turn an `overload_set<T>` into a `curried_overload_set<T>`. With a more encompassing idea of what a callable is, these tools become much easier to write.

§ 4. Acknowledgements

Thanks to Tim Song for many conversations about all the trouble this proposal gets itself into, and to Simon Brand for invaluable feedback.

§ 5. References

- [\[EWG65\]](#) "N3617 Lifting overload sets into function objects" by Philipp Juschka, 2015-05-22
- [\[N3617\]](#) "Lifting overload sets into function objects" by Philipp Juschka, 2013-03-14
- [\[N4128\]](#) "Ranges for the Standard Library, Revision 1" by E. Niebler, S. Parent, A. Sutton, 2014-10-10
- [\[P0119R2\]](#) "Overload sets as function arguments" by Andrew Sutton, 2016-05-28
- [\[P0356R3\]](#) "Simplified partial function application" by Tomasz Kamiński, 2017-11-09
- [\[P0382R0\]](#) "Comments on P0119: Overload sets as function arguments" by Tomasz Kamiński, 2016-05-29
- [\[P0551R3\]](#) "Thou Shalt Not Specialize std Function Templates!" by Walter E. Brown, 2018-03-16
- [\[P0573R2\]](#) "Abbreviated Lambdas for Fun and Profit" by Barry Revzin, Tomasz Kamiński, 2017-10-08
- [\[P0798R0\]](#) "Monadic operations for std::optional" by Simon Brand, 2017-10-06
- [\[P0834R0\]](#) "Lifting overload sets into objects" by Michael Dominiak, 2017-10-16
- [\[P0896R2\]](#) "The One Ranges Proposal" by Eric Niebler, Casey Carter, Christopher Di Bella, 2018-06-25
- [\[P0921R0\]](#) "Standard Library Compatibility Promises" by Titus Winters, 2018-02-01
- [\[abq.p0573\]](#) ABQ Wiki Notes, P0573R2 - November 2017
- [\[abq.p0834\]](#) ABQ Wiki Notes, P0834R0 - November 2017
- [\[andrzej.funcs\]](#) "Functions in std | Andrzej's C++ blog" by Andrzej Krzemieński, 2018-07-07
- [\[boost.hof\]](#) BOOST_HOF_LIFT - Boost.HigherOrderFunctions 0.6 documentation
- [\[f#.comp\]](#) Function associativity and composition | F# for fun and profit
- [\[over.call.object\]](#) Current Working Draft
- [\[point.free\]](#) Tacit programming - Wikipedia
- [\[rap.p0798\]](#) RAP Wiki Notes, P0798R0 - June 2018
- [\[winters.modern\]](#) "Modern C++ API Design: From Rvalue-References to Type Design" by Titus Winters, CppNow May 2018